

78. #include, et les onglets de la page

Le fichier voisin est versé ici, avant toute compilation.

1. Ce qu'il faut comprendre

`#include "dessins.cpp"` **verse le fichier voisin à cet endroit** avant que le compilateur ne voie quoi que ce soit. Il n'y a donc ni éditeur de liens, ni ordre de compilation à connaître.

Dans la page, les fichiers voisins sont des **onglets** : `#include` ne voit pas la différence. Et quand une faute tombe dans un fichier inclus, le message désigne **son** fichier et **sa** ligne — pas la ligne du texte assemblé, que personne n'a jamais vue.

Le programme ci-dessous tient en un fichier ; la leçon « Découper son programme » en montre un sur cinq.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **551 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
doubler()       4 o ← nombre() - ta fonction
main()          33 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          256 o ← CopierTuiles - 16 lettres sur 43
"DOUBLE DE 21"  12 o ← texte() - un texte gravé
```

Lettres gravées : BDELOU0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()  x 1   11 o sur place, dans main() + EcrireTexte (10 o)
nombre() x 1    8 o sur place, dans main() + doubler() (4 o), EcrireNombre (33 o)
image()  x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
doubler() x 1    7 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : affiche le double de 21.
//
// Ce programme tient dans un seul fichier. Un plus grand programme peut être
// découpé en plusieurs fichiers (les onglets de la page) : une ligne
// #include suivie du nom d'un fichier entre guillemets recopie ce fichier
// à cet endroit, avant la compilation.

#include <texte>    // écrit un texte à l'écran
#include <nombre>  // écrit un nombre en chiffres
```

2. UNE FONCTION — DOUBLER()

```
uint8_t doubler(uint8_t n) { return n * 2; } // Une fonction sur une seule ligne :
                                           // elle rend n fois 2.
```

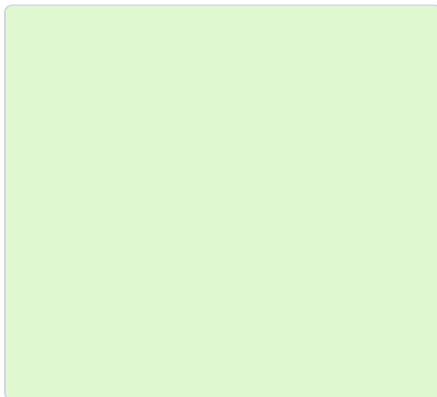
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    texte(2, 6, "DOUBLE DE 21");
    nombre(8, 8, doubler(21)); // 21 × 2 = 42 : affiche 042.

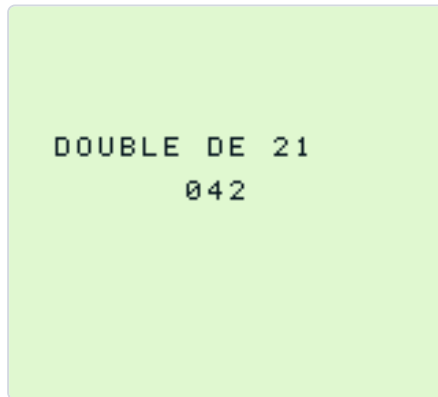
    while (true) { // Répète pour toujours :
        image();   // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« DOUBLE DE 21 », et 042 en dessous — la fonction a été appelée.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ le calcul est juste
- ✓ l’étiquette est là

Ensuite : **78.1. Chapitre 12 — Les fonctions : nommer un geste : ce qui vient** — chapitre 12 : Les fonctions : nommer un geste. ·
gameboy3 — écrire une cartouche Game Boy en C++